

Theory Evolution Auditor

中文版使用说明 · 案例驱动 · 含错误发现与纠正

像审查代码一样，审查理论如何演化。

本次主线案例

一个虚拟知识项目从 0 到 1，再从 1 到稳定发布。
它会经历：问题出现、理论诞生、边界膨胀、错误暴露、审查纠正、记录沉淀、稳定发布。
你会在案例中看到 TEA 的对象如何被拆开：Evolution Pair、Diff、Evidence、Decision、TER、Boundaries、Debt。

案例式教学

流程拆解

纠错演练

1. 这个 skill 要解决什么

把 “知识越来越多” 变成 “知识演化可审查、可纠错、可发布” 。

原始问题

AI 和人类一起协作久了，概念会变多、版本会变乱、旧假设会被悄悄改写，但没有一套像 Code Review 那样稳定的审查方式。

TEA 的回答

不生成理论，只审查理论是否 “正确地演化” 。重点不是内容有没有写得漂亮，而是变化有没有证据、边界有没有守住。

最终目标

让知识系统像软件一样：能回溯、能评审、能记录技术债、能决定什么时候收敛发布。

关键判断

TEA 关心四件事

- 有没有从 Base 到 Candidate 的清晰对照。
- 变化是不是能用证据指针说明。
- 有没有把边界讲清楚，而不是越界扩展。
- 遇到不确定时是否先拦住，而不是先放行。

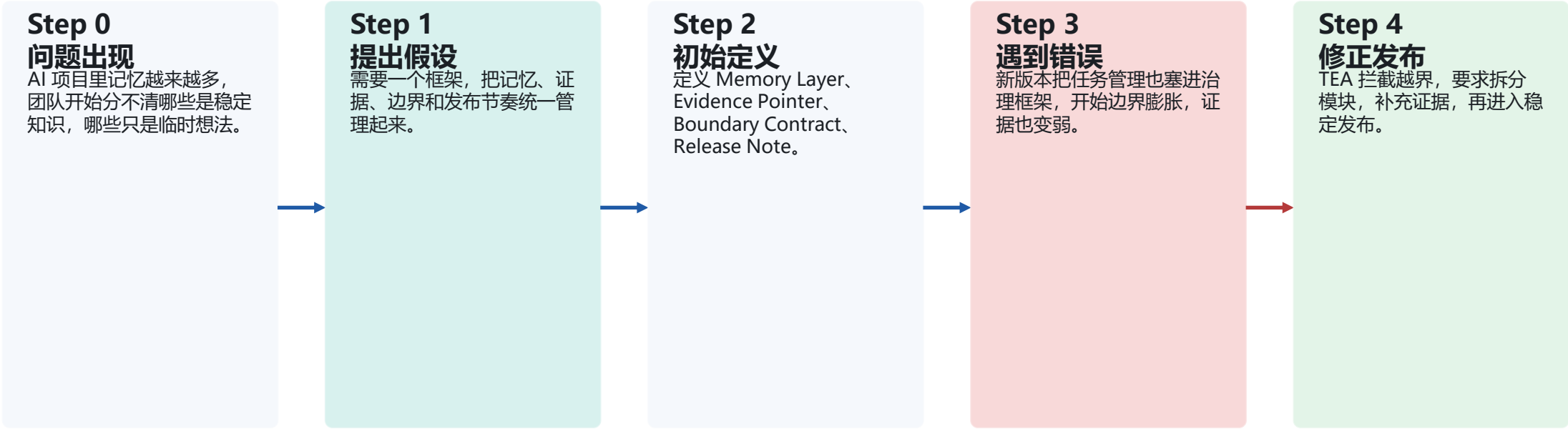
一句话

TEA = 理论演化的审计员，不是理论生成器。

它像 Code Review 一样工作，但审查对象不是代码，而是知识和概念的演化路径。

2. 主线案例：一个虚拟理论如何诞生

案例名：Agent Memory Governance Framework。它用来管理 AI 长期协作中的记忆、边界和证据。



这个案例的价值

它不是 “一个漂亮案例”，而是一个故意经历错误、审查、纠正的完整演化过程。这样才能把 TEA 的对象讲清楚。

3. 先看对象：TEA 里到底有哪些核心对象

这页是后面所有案例的“字典”。先认识对象，再看流程。

Evolution Pair

Base Version + Candidate Version，是审计输入的最小单元。

Diff

不是文本差异，而是理论语义上的变化。

Evidence Pointer

不是口头解释，而是可回指的证据链接/工单/报告。

Decision Contract

最终出口只有四种：PASS / PASS WITH DEBT / REVISE / BLOCK。

TER

把理论怎么变的写下来，方便下次复盘。

Boundaries

明确什么属于它，什么绝对不属于它。

后面每个案例都会反复出现这些对象，只是场景不同、证据不同、结论不同。

4. 主流程：Diff → Evidence → Decision

TEA 的骨架很简单，但每一步都不能省。



为什么这个流程有效

它强迫审查者先看“变化是什么”，再看“证据够不够”，最后再决定“放行还是拦截”。这能避免一上来就给结论。

最重要的纪律

没有 Evolution Pair，就没有审查；没有 Evidence Pointer，就不要假装已经验证；边界不清时，默认黄灯。

5. 案例一：项目健康度检查（模式 A）

从“当前状态”回头看整个演化链，判断是不是还能说清楚起点和边界。

输入

项目已有多个版本，团队想知道：这个理论是不是最初要解决的问题的延伸？有没有概念漂移？

审查动作

检查可回溯性、版本稳定性、闭环与证据、可积累性、概念漂移五个维度。

输出

健康评分、综合判断、建议行动。

示例结论

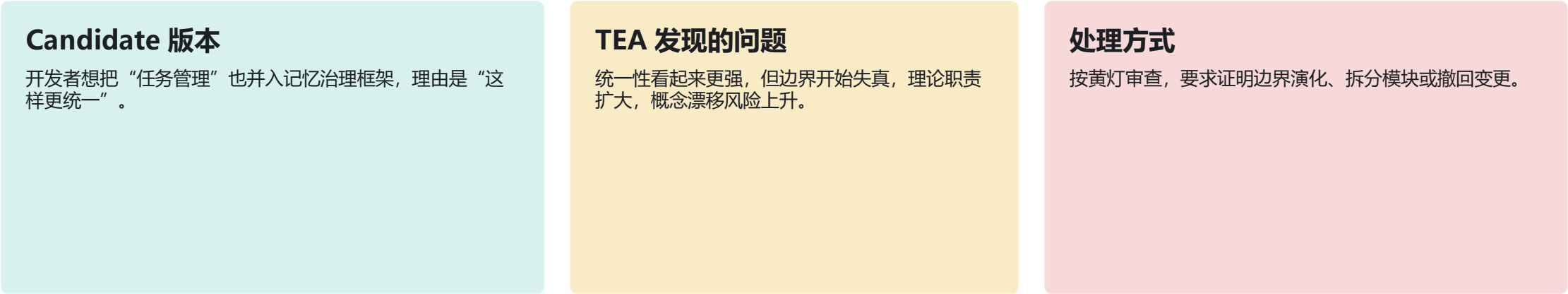


健康评分：3/5

当前仍能解释 Step 0，但证据链变弱，且边界出现歧义。建议补充 TER，并把超出边界的能力拆出去。

6. 案例二：PR 审查中的错误发现与纠正（模式 B）

这里加入一次“真实会发生的错误”：边界膨胀。TEA 的作用就是先把它拦下来。



这页的重点不是“发现错误”，而是“发现后怎么纠正并重新进入稳定路径”。

7. 错误是怎么被拆开的：对象级纠错

把“边界膨胀”拆成可判断的对象，而不是一句“感觉不对”。

错误信号

1. 新概念开始堆叠。
2. 功能归属变模糊。
3. 证据开始只剩口头解释。
4. 团队说“先统一再说”。

TEA 的拆解

- Evolution Pair: Base 是“记忆治理”，Candidate 变成“记忆 + 任务”。
- Diff: 不只是新增能力，而是职责扩大。
- Boundaries: 原本明确的 out-of-scope 被悄悄吞掉。
- Evidence: 没有证据证明任务管理必须属于同一理论。

纠正后的结果

把任务管理拆成独立模块，保留记忆治理的核心边界；如果需要联合工作，通过接口协作而不是概念合并。

结论

REVISE: 需要修改后重新提交，不是直接放行。

8. 案例三：TER 如何记录 “这个理论是怎么变出来的”

把演化过程写成可复盘的记录，后续审查才不会靠记忆猜。



TER 的价值

- 它不是项目日志，而是理论演化的记忆装置。下次再审查时，Base 不靠回忆，靠记录。

9. 应用方式不止一种：同一个 skill 的四种用法

同一套理念，可以在不同场景里以不同强度使用。

手动审查

你在聊天里描述当前状态，TEA 帮你判断健康度或阶段。

PR 门禁

每次重构或新增概念前，先过黄灯/红灯判断。

Release 门禁

只有 PASS / PASS WITH DEBT 才能进入稳定版本。

长期治理

用 TER 和 Boundaries 让知识系统可持续演化。

推荐使用顺序

- 先手动审查，熟悉对象和判断方式；再把它固化到 PR 或 Release 门禁；最后把 TER 作为长期记忆层。

10. 信息不足时，TEA 为什么要先停下来

这是它最重要的安全感来源：不知道就不要装知道。

危险做法

只给一个模糊描述，就直接产出完整审查报告，甚至还假装有证据。

TEA 的做法

先问问题：当前变更是什么？有没有 TER？最初问题是什么？边界在哪里？

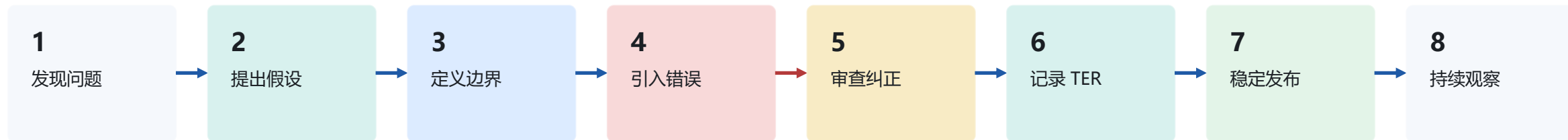
结果

减少幻觉，减少错放行，减少后续返工。

Fail-safe 不是保守，它是在证据不足时，主动保护理论的边界完整性。

11. 从创建到收口：一条完整的演化链

把虚拟案例的全生命周期压成一张图，方便讲给别人听。



这条链背后的教学点

- TEA 不是在“证明自己正确”，而是在训练一个知识系统如何在出错后继续正确地演化。

12. 你在这个 skill 里会看到的结论风格

不同场景会输出不同结论，但语言是统一的。

PASS

演化健康，可以放行。

PASS WITH DEBT

可以放行，但要记一笔技术债。

REVISE

需要修改后重新提交。

BLOCK

核心边界有问题，必须拦截。

如果你希望这个 skill 更偏“教程”，我建议每个案例最后都落到一个明确的 Decision。这样观众会更容易学会怎么用。

收束一下

一个虚拟理论从创建、犯错、纠错，到稳定发布的全过程。

你应该记住什么

TEA 的重点不是“更聪明地生成”，而是“更严谨地演化”。它通过对象、流程、边界和决策契约，把知识系统做成可审查的东西。

这份 PPT 的用法

可以拿去讲原理、讲案例、讲流程，也可以作为你后续扩展 skill 的模板。

最后一句

让理论像代码一样被审查，让演化像发布一样被记录。

Decision: PASS